

Докса и логос: операторная модель взаимодействия языковой модели и формальной системы

Владимир Лапшин*

Аннотация

Большие языковые модели хорошо переводят текст на формальный язык, но не умеют рассуждать корректно: они не различают правильный шаг от правдоподобного, не отслеживают, на чём держится ответ, и не отказываются от догадок. Символические решатели умеют всё это, но не умеют читать текст. Мы предлагаем рассматривать это разделение не как инженерный компромисс, а как эпистемологическое различие *доксы* и *логоса*: докса — источник предложений, не имеющих вывода внутри системы; логос — формальная система, которая принимает или отвергает их по строгим правилам. Мы вводим семейство операторов взаимодействия доксы и логоса и различаем в нём три роли: внешние (доксальные) поставляют предложение, невыводимое из теории; внутренние (логосные) вычисляют реакцию системы; протокольные фиксируют, классифицируют и контролируют обмен. Главная мысль: этот интерфейс асимметричен. Мы показываем, что различие имеет строгие формальные аналоги — доксистическая логика, немонотонные логики, абдукция, — и что каждый из них описывает одну из операций взаимодействия. Мы также описываем работающую систему Ankyra, где большая языковая модель играет роль доксы, а символьный движок — роль логоса.

1. Введение

Формальные системы имеют фундаментальный предел. Теоремы Гёделя о неполноте показали, что любая достаточно богатая непротиворечивая система содержит утверждения, которые она не может ни доказать, ни опровергнуть [1]. Проблема остановки Тьюринга показала, что не существует общего алгоритма, ко-

*merfill@yandex.ru

торый для любой программы и входа определял бы, завершится ли она [2]. Эти результаты обычно интерпретируются как абсолютные границы познания.

Мы предлагаем другую интерпретацию. Это границы *статических* систем — замкнутых множеств аксиом и правил вывода. Но человеческое мышление не является статической системой. Оно не начинается с фиксированного набора аксиом и не выводит следствия до упора. Оно постоянно расширяет, пересматривает и достраивает свои формализмы, опираясь на то, что античная традиция называла *доксой*.

Различение доксы и логоса имеет долгую историю. Уже у Ксенофана (VI в. до н. э.) *докса* ($\delta\acute{o}\xi\alpha$) обозначает *мнение или убеждение*, принципиально противопоставляемое достоверному знанию (истине). Парменид (V в. до н. э.) противопоставляет *δόξα истине* ($\alpha\lambda\eta\theta\epsilon\iota\alpha$): докса — это «обманчивый порядок» чувственного мира, тогда как истина доступна только *логосу* — чистому мышлению, не доверяющему свидетельствам глаз и ушей [3]. Именно у Парменида различение данных чувств и данных логоса становится радикальным: только второе, по его мысли, реально и истинно.

Платон радикализировал это различение. У него $\delta\acute{o}\xi\alpha$ — это познание *того, что кажется*, тогда как *эпистеме* ($\epsilon\pi\iota\sigma\tau\acute{\eta}\mu\eta$) — познание *того, что есть*. Докса может быть истинной или ложной, но она всегда остаётся *неустойчивой, конечной, случайной* — она «выражается отношением к предмету, а не обоснованием истины» [4]. Эпистеме же — это знание, *сопровожаемое логосом*, то есть объяснением, обоснованием, *строгим рассуждением*.

В «Теэтете» Платон обсуждает и в конечном счёте отвергает определение знания как «истинного мнения с логосом». Это важно для нас: логос не просто «добавляется» к доксе, он действует иначе.

Аристотель затем уточнил: докса — это *мнение*, которое может быть истинным, но не имеет *необходимого* характера; эпистеме — это знание *необходимого*, добытое через *доказательство* [5].

Таким образом, в античной традиции сложилось устойчивое различение: *докса* — это мнение, опыт, вероятностное суждение, укоренённое в чувственном и культурном; *логос* (или *эпистеме*) — это строгое, обоснованное, проверяемое знание. Докса *шире*, но *менее надёжна*. Логос *уже*, но *гарантирован*.

Прежде чем отождествлять эти термины с чем-либо в системе, зафиксируем, чем докса отличается от логоса. Не через отрицание («докса — это не формализованное знание»), а через положительные характеристики.

- **Историчность.** Опыт накапливается слоями: новый не отменяет старый, а

ложится поверх. Логос расширяется добавлением аксиом, но не слоится.

- **Ассоциативность.** Докса связывает разнородное — видит сходство там, где формально его нет. Логос связывает только то, что связано правилами вывода.
- **Непосредственность.** Докса даёт ответ без вывода, до проверки. Логос приходит к ответу только через вывод.
- **Контекстуальность.** Значение в доксе определяется контекстом. В логосе значение зафиксировано языком L .
- **Вероятностность.** Докса даёт степень уверенности, а не бинарность. Логос даёт «доказано» или «не доказано», без середины.

Это характеристики типа знания, а не его носителя. Они не утверждают ничего о том, как докса устроена «на самом деле»; они фиксируют, чем она *отличается* от формальной системы, если рассматривать её как источник предложений.

В этой работе мы используем в роли доксы большую языковую модель. Это не метафора и не аналогия: LLM удовлетворяет всем пяти характеристикам.

- **Историчность.** LLM обучена на накопленном человеческом тексте, и её веса несут следы множества эпох и жанров. Дообучение добавляет слои, не стирая предыдущие.
- **Ассоциативность.** Механизм внимания связывает токены по статистической близости, а не по правилам вывода. Модель видит сходство там, где формально его нет.
- **Непосредственность.** Ответ порождается за один прямой проход, без явного вывода и без промежуточной проверки.
- **Контекстуальность.** Значение токена зависит от всего окна контекста; одно и то же слово в разных контекстах получает разное представление.
- **Вероятностность.** Выход модели — распределение вероятностей по следующему токenu, а не бинарный вердикт.

Именно это соответствие делает отождествление осмысленным. Если бы LLM не обладала этими характеристиками, использовать её в роли доксы было бы нельзя, и различие осталось бы философской метафорой. Оно им не является.

Почему такое отождествление становится возможным только теперь. Формальная сторона разрабатывалась веками: логика, теория доказательств, решающие процедуры, языки спецификаций. Логос был и остаётся предметом точного исследования. Докса, напротив, оставалась без инструмента: её описывали фило-

софски, но не могли к ней обратиться машинно. Большая языковая модель — первый инструмент такого рода. Именно поэтому различие, известное две с половиной тысячи лет, только теперь становится *архитектурным принципом*, а не философской метафорой.

Логос в нашей системе — это символический решатель: формальная система, которая принимает или отвергает предложения доксы по строгим правилам. Различение доксы и логоса не означает, что одно отменяет другое. Человеческое мышление опирается на оба источника: докса поставяет содержание, логос проверяет и удерживает. В этой работе нас интересует не природа доксы как таковой, а *интерфейс* между ней и формальной системой.

Ключевой тезис работы состоит в следующем. Хотя докса не является теорией в языке формальной системы, *функции её взаимодействия с этой системой* могут быть описаны формально. Мы вводим семейство операторов взаимодействия и различаем в нём три роли: *внешние* (доксальные) операторы поставяют предложение, невыводимое из теории; *внутренние* (логосные) вычисляют реакцию системы на это предложение; *протокольные* фиксируют, классифицируют и контролируют обмен. Судьбу предложения решает логос — он *не производит* его сам.

Статья устроена следующим образом. В разделе 2 мы разбираем теоретические основы: различие доксы и логоса в античной традиции и в современной логике, обзор доксистической логики, немонотонных логик, абдукции и динамических логик убеждений. В разделе 3 вводим семейство операторов взаимодействия доксы и логоса и его формальные свойства. В разделе 4 описываем конкретную реализацию этой модели. В разделе 5 обсуждаем границы подхода, его связь с гипервычислениями и практические импликации для объяснимого ИИ. Раздел 6 завершает статью.

2. Теоретические основы

В разделе 1 мы ввели различие доксы и логоса и показали его античные истоки. Здесь мы не будем повторять историю. Наша задача — показать, что это различие имеет *строгие формальные аналоги* в современной логике и что эти аналоги *релевантны* для нашей модели.

Мы рассмотрим три подхода: доксистическую логику, немонотонные логики и абдукцию. Каждый схватывает отдельный аспект взаимодействия доксы и логоса. Ни один из них не описывает доксу как таковую — это и не требуется: нас интересует не природа доксы, а те её функции, через которые она расширяет формальную систему. Именно эти функции мы и описываем в разделе 3.

2.1. Доксастическая логика

Термин «доксастическая логика» (doxastic logic) был введён Хинтиккой в 1962 году для формального моделирования *мнений и убеждений* ($\delta\acute{o}\xi\alpha$) агента, в отличие от знания ($\epsilon\pi\iota\sigma\tau\acute{\eta}\mu\eta$) [6]. Основным оператор B_α читается как «Альфа считает, что...». Доксастическая логика занимается *синхроническими* принципами — тем, как убеждения связаны между собой в данный момент.

Но она *молчит* о том, как убеждения меняются во времени. Это ограничение было осознано, и ответом на него стала *динамическая доксастическая логика* (DDL), введённая Кристером Сегербергом в 1990-х годах [7]. DDL добавляет к статической доксастической логике *модальные операторы изменения убеждений*: $[*p]q$ читается как «q выполняется после ревизии убеждений на p» [8].

Таким образом, DDL — это формальный аналог не доксы как таковой, а *оператора ревизии формальной системы* при поступлении нового предложения от доксы. Это именно то, что мы называем *функцией доксы*.

2.2. Немонотонные логики

Классическая логика *монотонна*: если $\Gamma \vdash \phi$, то $\Gamma \cup \Delta \vdash \phi$. Добавление новых аксиом *не отменяет* ранее выведенных следствий. Но формальная система, взаимодействующая с доксой, работает иначе. Новый опыт может *отменить* старые выводы. Если я знаю, что Твити — птица, я предполагаю, что он может летать. Если позже я узнаю, что Твити — пингвин, я *отменяю* вывод «Твити летает», хотя раньше он следовал из «Твити — птица».

Немонотонные логики — это формальные системы, которые моделируют именно эту *отменяемость* выводов. Классические подходы включают:

- **Логика по умолчанию** (Default Logic) *Рейтера*: правила вида «если A и нет информации об исключениях, то B », которые могут быть отменены при появлении исключения [9].
- **Циркумскрипция** (Circumscription) *Маккарти*: минимизация «ненормальных» ситуаций, позволяющая выводить «по умолчанию» [10].
- **Автоэпистемическая логика** *Мура*: моделирование убеждений агента о его собственных убеждениях [11].

Все эти формализмы признают, что *выводы формальной системы не являются окончательными*. Они *отменяемы* при поступлении новой информации из доксы. И это не «недостаток». Это *свойство* системы, которая *открыта* для взаимодействия с доксой. Немонотонные логики формализуют это свойство, но не

устраняют его. Они показывают, как система может *работать* с отменяемыми выводами, не теряя когерентности.

2.3. Абдукция

Если немонотонные логики описывают *отмену* выводов, то *абдукция* описывает *порождение* новых гипотез. Термин был введён Чарльзом Сандерсом Пирсом (1839–1914), который различал три типа вывода: дедукцию, индукцию и *абдукцию* — «творческое порождение объяснительных гипотез» [12].

Пирс характеризовал абдукцию как рассуждение «от следствия к причине». Если мы наблюдаем факт Q , и знаем, что $P \rightarrow Q$, то абдукция позволяет *предположить* P как *возможное объяснение* Q . Это не дедукция (где P необходимо следует) и не индукция (где P вероятно следует). Это *порождение гипотезы*, которая объясняет наблюдение [13].

В современной философии науки абдукция часто отождествляется с «выводом к наилучшему объяснению» (Inference to the Best Explanation, IBE). Однако, как отмечают исследователи, *отождествление абдукции и IBE не является строгим*. Пирс различал *порождение* гипотезы (абдукция в узком смысле) и *выбор* наилучшей из них (IBE). Для нашей модели важно именно *порождение*: докса *предлагает* гипотезы, а формальная система *проверяет* их [14].

Абдукция *формализуема* — но только при условии, что формализуем весь «паттерн» абдуктивного вывода, включая правила порождения и обоснования гипотез. Существуют формальные системы, где абдукция добавлена к логике высказываний как *внешний оператор расширения* [15]. Это означает, что абдукция может быть описана не как правило вывода внутри логики, а как операция над теорией, поставляющая в неё новое предложение.

2.4. ДСМ-метод: синтез познавательных процедур

Абдукция описывает *порождение* гипотезы. Отдельный вопрос — как гипотеза *принимается* в качестве рабочей, если она не выводится из теории. Формальный подход к этому вопросу предложен В. К. Финном в рамках *ДСМ-метода автоматического порождения гипотез* [29]. Метод назван в честь Джона Стюарта Милля и формализует его индуктивные методы средствами современной логики.

Ключевая идея ДСМ-метода состоит в том, что познавательная процедура не сводится ни к индукции, ни к аналогии, ни к абдукции по отдельности. Она представляет собой их *синтез*: индукция порождает гипотезы о причинно-следственных связях на основе сходства и различия объектов; аналогия пере-

носит эти гипотезы на новые объекты; абдукция проверяет условие каузальной полноты — достаточно ли найденных причин для объяснения наблюдаемого эффекта [30, 31].

Для нашей модели ДСМ-метод важен по двум причинам. Во-первых, он показывает, что *порождение* и *принятие* гипотезы — разные операции, и вторая не сводится к первой. Во-вторых, гипотеза в ДСМ-методе получает *степень правдоподобности*, а не бинарный статус «истинно/ложно», что соответствует вероятностной природе доксы (раздел 1). ДСМ-метод можно рассматривать как ещё один формальный аналог *внешнего* оператора: он описывает, как гипотеза входит в теорию с явно указанным основанием.

2.5. Границы и сложность

Четыре подхода описывают разные операции над формальной теорией. Доксастическая логика и DDL работают с убеждениями агента как с формализованным срезом; немонотонные логики описывают отмену выводов при поступлении новой информации; абдукция поставяет в теорию новое предложение; ДСМ-метод порождает гипотезы и оценивает их правдоподобность через синтез индукции, аналогии и абдукции. Ни один из них не работает с доксой как с объектом: все четыре имеют дело с операциями, а не с источником. Именно эти операции мы и называем *функциями доксы*.

С этими операциями связана отдельная трудность. Немонотонные логики вычислительно сложнее классической [16], и для многих из них задача проверки следования неразрешима или имеет очень высокую сложность [17]: не существует общего алгоритма, который для любой теории и цели определил бы, является ли цель следствием. Процедуру построить можно, но гарантировать её завершение — нельзя. Абдукция порождает множество гипотез, и выбор между ними — отдельная задача без формального критерия истинности: пропозициональная абдукция Σ_2^P -полна [18], а логическая абдукция в общем случае вычислительно трудна [19]. ДСМ-метод, будучи синтезом трёх познавательных процедур, наследует их сложность: проверка каузальной полноты и оценка правдоподобности гипотез требуют перебора подмножеств признаков и в общем случае не сводятся к полиномиальным процедурам [29].

2.6. Связь с нашей моделью

Функции доксы — то, как она расширяет формальную систему — мы описываем формально. Каждый из четырёх подходов выше соответствует одному или нескольким операторам, и каждый оператор принадлежит одной из трёх ролей:

- **Доксастическая логика** → оператор *фиксации*: что агент «держит» как убеждения. Это *протокольная* функция среды, реализуемая журналом.
- **Динамическая доксастическая логика** → оператор *ревизии формальной системы*. Это *внутренний* оператор: *логос* вычисляет, как теория меняется при поступлении новой информации из доксы.
- **Немонотонные логики** → оператор *отмены*. Это *внутренний* оператор: *логос* заново вычисляет, что следует, при появлении исключений, полученных из доксы.
- **Абдукция и ДСМ-метод** → оператор *порождения гипотез*. Это *внешний* оператор: докса поставяет гипотезу, которую формальная система проверяет. Абдукция описывает само порождение; ДСМ-метод добавляет к нему оценку правдоподобности через синтез индукции, аналогии и абдукции.

Это не полный список. Мы вернёмся к нему в разделе 3, где введём *семейство операторов взаимодействия доксы и логоса* и формальные определения ролей. Уже сейчас видно: различие доксы и логоса опирается на формальные аналоги в современной логике, и эти аналоги можно использовать для построения работающей системы.

3. Семейство операторов взаимодействия доксы и логоса

В разделе 2 мы показали, что различие доксы и логоса опирается на формальные аналоги в современной логике. Здесь мы вводим *семейство операторов*, через которые докса и логос взаимодействуют. Нас интересует не содержание доксы, а *протокол* её использования.

Раздел устроен следующим образом. Сначала мы даём основные определения и критерий, по которому операторы делятся на три класса (раздел 3.1). Затем описываем каждый класс по отдельности: внешние операторы, внутренние и протокольные. После этого вводим роль Арбитра — внешней инстанции, которая не принадлежит ни доксе, ни логосу. Наконец, формулируем свойства семейства, описываем процесс рассуждения как последовательность актов обращения к доксе и уточняем роль логоса в этом процессе.

Главная мысль раздела в том, что этот протокол *асимметричен*: докса вносит содержание предложений, тогда как всё, что решает, фиксирует и контролирует их судьбу, принадлежит логосу и среде взаимодействия.

3.1. Основные определения и критерий роли

Пусть $F = (A, R, L)$ — формальная система с аксиомами A , правилами вывода R и языком L , и пусть $\text{Cn}(F)$ — множество её следствий. Пусть D — докса (раздел 1): источник предложений для F , не являющийся её частью и не образующий теорию в L .

Оператор может расширять *теорию* (новые аксиомы, правила или объекты) или сам *язык*; изменение L меняет то, что считается формулой, и эти два вида расширения нельзя смешивать. В дальнейшем мы рассматриваем только расширения теории: язык L считается фиксированным.

Пусть $\Sigma = \langle F, \mathcal{J}, \Pi \rangle$ — состояние системы, где F — теория, $\mathcal{J}$ — журнал, Π — последовательность предложенных, но ещё не принятых предложений. Оператор — это функция $o : \Sigma \times X \rightarrow \Sigma$, где X — вход (наблюдение, новая информация, контекст или пустой вход). Классификация ведётся по тому, что именно оператор меняет в Σ : теорию, очередь предложений или журнал.

- **Внешний** (доксальный) оператор *производит предложение*: содержание, источник которого — докса. Формально: $o(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F, \mathcal{J}, \Pi \cup \langle P \rangle \rangle$. Теория и журнал не меняются; предложение попадает в Π и ждёт решения логоса.
- **Внутренний** (логосный) оператор *производит новую теорию*: F' , вычисленную из F и входа. Формально: $o(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F', \mathcal{J}, \Pi \rangle$, где F' — детерминированная функция от F и x при фиксированных правилах. Ни журнал, ни очередь Π оператор не меняет: извлечение входа из Π и запись об акте — работа протокольного оператора (раздел 3.4).
- **Протокольный** оператор *извлекает предложение и производит запись*: он не изменяет F . Формально: $o(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F, \mathcal{J}', \Pi' \rangle$, где $\mathcal{J}' = \mathcal{J} \cup \{\text{запись об акте}\}$, а Π' — очередь Π без извлечённого предложения x .

Класс	Состав	Что меняет в Σ
Внешние (доксальные)	$O_{abd}, O_{anal}, O_{int}$	добавляют предложение в Π ; F и $\mathcal{J}$ не меняются
Внутренние (логосные)	$O_{rev}, O_{def}, O_{spec}, O_{ctx}$	вычисляют новую теорию F' из F и входа; $\mathcal{J}$ и Π не меняются
Протокольные (среда)	O_{fix}	дополняют журнал $\mathcal{J}$; F и Π не меняются

3.2. Внешние операторы (доксальные)

Внешние операторы не меняют ни теорию F , ни журнал $\mathcal{J}$; они добавляют предложение в очередь Π , откуда его позже забирает логос. Запись об акте делает протокольный оператор (раздел 3.4).

Оператор внесения гипотезы (o_{abd}). Берёт состояние $\langle F, \mathcal{J}, \Pi \rangle$ и предложенную доксой пару $\langle Q, P \rangle$ — наблюдение Q и гипотезу P , объясняющую Q , — и добавляет P в конец Π :

$$o_{abd}(\langle F, \mathcal{J}, \Pi \rangle, \langle Q, P \rangle) = \langle F, \mathcal{J}, \Pi \frown \langle P \rangle \rangle,$$

где выполнены четыре условия: $P \notin \text{Cn}(F)$ (гипотеза нова), $Q \notin \text{Cn}(F)$ (наблюдение не выводится из теории и потому требует объяснения), $Q \in \text{Cn}(F \cup \{P\})$ (гипотеза объясняет наблюдение) и $F \cup \{P\}$ непротиворечиво.

Условие непротиворечивости существенно: если $F \cup \{P\}$ противоречиво, то по принципу взрыва из него следует любое Q , и условие объяснения становится тривиальным. Условие $Q \notin \text{Cn}(F)$ столь же существенно с другой стороны: если наблюдение уже выводится из F , абдуктивное введение P ничего не объясняет.

Условие объяснения само по себе не требует *минимальности* гипотезы: если P объясняет Q , то и $P \wedge R$ объясняет Q . Отбор гипотез без избыточных конъюнктов выполняется на этапе протокольной классификации (раздел 3.10).

Теория не меняется; предложение попадает в Π и ждёт решения логоса. Это формальный аналог абдукции: гипотеза *предлагается*, а не выводится [15].

Оператор аналогии (o_{anal}). Берёт состояние $\langle F, \mathcal{J}, \Pi \rangle$ и пару областей $\langle F_s, F_t \rangle$ — знакомую F_s и новую F_t — и добавляет в Π предложение P_{analog} , переносящее структуру F_s на F_t :

$$o_{anal}(\langle F, \mathcal{J}, \Pi \rangle, \langle F_s, F_t \rangle) = \langle F, \mathcal{J}, \Pi \cup \{P_{analog}\} \rangle,$$

где $P_{analog} \notin \text{Cn}(F)$. Выбор структурного отображения внешний; логос лишь проверяет, сохраняется ли оно.

Оператор интуиции (o_{int}). Берёт состояние $\langle F, \mathcal{J}, \Pi \rangle$ и цель ϕ и добавляет в Π ответ ϕ' , который не следует из F :

$$o_{int}(\langle F, \mathcal{J}, \Pi \rangle, \phi) = \langle F, \mathcal{J}, \Pi \cup \{\phi'\} \rangle,$$

где $\phi' \notin \text{Cn}(F)$. Это оператор, ближе всего стоящий к оракулу. Докса даёт ответ; логос проверяет его, если может, иначе откладывает как гипотезу. Оракулоподобен здесь источник ответа, а не носитель, который вполне может быть вычисли-

мым.

3.3. Внутренние операторы (логосные)

Эти операторы часто принимают за функции доксы, потому что они запускаются её входом. На самом деле их вычисляет логос; вход — лишь их аргумент. Внутренние операторы меняют теорию F , но не трогают журнал $\mathcal{J}$: запись об акте делает протокольный оператор (раздел 3.4).

Вход внутреннего оператора приходит из одного из двух источников. Первый — *предложение, извлечённое из очереди* Π протокольным оператором: протокол удаляет его из Π , фиксирует акт в журнале и передаёт предложение внутреннему оператору как аргумент. Так происходит при ревизии и отмене. Второй источник — вход, уже присутствующий в F (конфликтующие правила при выборе специфичности), или *объявленный протоколом* (контекст); в этом случае Π не участвует. Во всех случаях сам внутренний оператор меняет только F ; изменение Π и $\mathcal{J}$ — прерогатива протокола.

Оператор ревизии (o_{rev}). По состоянию $\langle F, \mathcal{J}, \Pi \rangle$ и новой информации p , поступившей из очереди Π , возвращает F' , в которой убеждения пересмотрены с учётом p :

$$o_{\text{rev}}(\langle F, \mathcal{J}, \Pi \rangle, p) = \langle F', \mathcal{J}, \Pi \rangle,$$

где F' — детерминированная функция от F и p при фиксированной функции отбора, реализующей *ревизию базы знаний* (belief base revision) в духе принципов AGM [20]. Функция отбора считается *частью формальной системы* F : она входит в её структуру наравне с аксиомами и правилами вывода и не является скрытым параметром состояния Σ . Тем самым F здесь — база аксиом и правил вместе с порядком предпочтения, а не логически замкнутое множество убеждений; поэтому речь идёт именно о ревизии базы. Докса поставляет p ; логос решает, как p влияет на F . Извлечение p из Π и запись об акте выполняет протокольный оператор (раздел 3.4); внутренний оператор меняет только F .

Оператор отмены (o_{def}). По состоянию $\langle F, \mathcal{J}, \Pi \rangle$ и исключению e , поступившему из очереди Π , возвращает F' , в которой ранее выведенные следствия отменены:

$$o_{\text{def}}(\langle F, \mathcal{J}, \Pi \rangle, e) = \langle F', \mathcal{J}, \Pi \rangle.$$

Это формальный аналог немонотонных логик. Отмена не означает удаления аксиом: то, что было выводимо, перестаёт быть выводимым при новой конфигурации. Извлечение e из Π и запись об акте выполняет протокольный оператор.

Оператор специфичности (o_{spec}). По состоянию $\langle F, \mathcal{J}, \Pi \rangle$ и двум правилам r_1, r_2 , присутствующим в F , возвращает F' , в которой выбрано более специфичное:

$$o_{\text{spec}}(\langle F, \mathcal{J}, \Pi \rangle, \{r_1, r_2\}) = \langle F', \mathcal{J}, \Pi \rangle.$$

Специфичность — это отношение, *вычисляемое логосом* по иерархии F ; докса его не поставляет. Это формальный аналог предпочтения по специфичности в немонотонных логиках. Поскольку правила уже в F , очередь Π не меняется.

Оператор применения контекста (o_{ctx}). По состоянию $\langle F, \mathcal{J}, \Pi \rangle$ и объявленному контексту Γ возвращает F' , в которую добавлены аксиомы контекста Γ :

$$o_{\text{ctx}}(\langle F, \mathcal{J}, \Pi \rangle, \Gamma) = \langle F \cup \Gamma, \mathcal{J}, \Pi \rangle.$$

Контекст *объявляется* протоколом; его *применение* внутренне. Поскольку Γ не является предложением из очереди, Π не меняется.

3.4. Протокольные операторы (среда)

Эти функции не принадлежат ни доксе, ни логосу. Они не вносят содержания и не меняют теорию F ; они меняют только журнал $\mathcal{J}$ и тем самым делают обмен корректно определённым, детерминированным и проверяемым.

Строго говоря, оператором в смысле раздела 3.1 является только фиксация: она имеет вид $o : \Sigma \times X \rightarrow \Sigma$. Остальные протокольные функции — классификация, объявление, проверки заземления, волновой протокол — это правила и процедуры, задающие, как именно происходит запись. Общий вид операторной части:

$$o_{\text{fix}}(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F, \mathcal{J}', \Pi \rangle, \quad \mathcal{J}' = \mathcal{J} \cup \{\text{запись об акте}\}.$$

Журнал и фиксация (o_{fix}). Каждый шаг фиксируется в журнале: идентификатор оператора, волна, обоснование, источник и результат (принято, отвергнуто, отложено). Журнал — это протокол взаимодействия, не часть ни доксы, ни логоса. Фиксация означает выбор формализованного среза доксы, к которому обращаются, и запись этого акта; это учётное действие среды, а не функция доксы.

Детерминированная классификация. Каждое предложение из Π классифицируется по фиксированным правилам (см. раздел 3.10), независимо от «настроения»; именно это делает обмен воспроизводимым.

Объявление контекста и фрагмента. Контекст Γ или требуемый фрагмент логики *объявляется* параметром запуска; его применение внутренне (см. o_{ctx} выше).

Проверки заземления. Проверки дословных цитат и другие структурные проверки удостоверяют, что предложение закреплено в источнике.

Волновой протокол. Правило «не более одного предложения за акт обращения» делает каждый акт проверяемым и оставляет след того, где и почему изменилась теория.

3.5. Принятие предложений

Множество Π содержит предложения, ожидающие решения. Решение принимает логос через протокольную классификацию (раздел 3.10). Возможны три исхода.

Принято. Протокол извлекает предложение из Π и передаёт его соответствующему внутреннему оператору. Тот применяет предложение к F ; если предложение опирается на цитату из источника, оно входит в F как аксиома; если это допущение, оно входит с меткой H и учитывается при оценке обоснованности ответа. Протокол фиксирует акт в журнале $\mathcal{J}$.

Отклонено. Протокол извлекает предложение из Π , не передаёт его внутреннему оператору и фиксирует отказ в журнале с кодом причины. Теория F не меняется.

Отложено. Предложение остаётся в Π до следующей волны или до исчерпания бюджета.

Во всех трёх случаях протокол отвечает за извлечение предложения из Π и за запись в $\mathcal{J}$, а внутренний оператор — за изменение F . Журнал хранит историю: какое предложение было рассмотрено и какое решение принято. Само Π истории не хранит: оно содержит только ожидающие предложения.

3.6. Арбитр

До сих пор мы описывали операторы, которые поставляют, вычисляют и фиксируют. Но остаётся вопрос: кто решает, когда процесс останавливается, какое из конкурирующих предложений доксы принимается и достигнута ли цель? Этот вопрос требует введения отдельной роли.

Арбитр — это внешняя инстанция, которая не принадлежит ни доксе, ни логосу, но для которой система существует. Арбитр не производит содержания и не вычисляет реакцию логоса; он *выбирает*, какой оператор из семейства применить к текущему состоянию.

Роль Арбитра моделируется как внешняя по отношению к системе функция выбора $W : \Sigma \rightarrow \mathcal{O}$, где Σ — состояние системы, а $\mathcal{O}$ — множество операторов семейства. Тип функции задан, но её алгоритмическая реализация находится вне

формальной модели Σ — иначе мы получили бы бесконечную регрессию (Арбитр сам подлежал бы проверке, и требовался бы мета-Арбитр).

Арбитр может быть:

- *человеком* — исследователем, пользователем, экспертом, который принимает решения там, где формальный критерий отсутствует или недостаточен;
- *внешней программой* — тестовым раннером, верификатором, оракулом, который возвращает объективный вердикт (например, прошёл ли тест);
- *композицией* — например, автоматический арбитр принимает решения в известных случаях, а человек подключается только тогда, когда автоматический воздерживается.

Модель намеренно не автоматизирует шаг Арбитра полностью. Никакой формальный критерий не решает спор между несовместимыми предложениями доксы, если оба совместимы с теорией. Выбор остаётся за тем, кто за него отвечает, — человеком или внешней программой, действующей по фиксированному критерию.

3.7. Свойства семейства

Внешние операторы не коммутируют, и порядок их применения существенен. Разберём это на примере. Пусть теория F содержит $\text{is_a}(\text{tweety}, \text{bird})$ и умолчание $\text{bird} \Rightarrow \text{fly}$, а докса предлагает $\text{is_a}(\text{tweety}, \text{penguin})$.

Порядок первый: сначала o_{abd} , затем принятие предложения, затем o_{spec} . После o_{abd} предложение $\text{is_a}(\text{tweety}, \text{penguin})$ попадает в Π и ждёт решения. Если логос его принимает, оно входит в F , и в теории появляется более специфичное умолчание $\text{penguin} \Rightarrow \neg \text{fly}$. Оператор o_{spec} выбирает его, и fly отменяется.

Порядок второй: сначала o_{spec} , затем o_{abd} . В теории пока одно умолчание ($\text{bird} \Rightarrow \text{fly}$), конкурента нет, и выводится fly . Только после этого o_{abd} предлагает $\text{is_a}(\text{tweety}, \text{penguin})$. Теперь, если предложение принято, в теории появляется второе умолчание, которое конфликтует с уже сделанным выводом. Это вынуждает не выбор специфичности, а *ревизию*: ранее выведенное следствие пересматривается.

Одни и те же операторы в разном порядке дают разные теории: процесс историчен, и каждый шаг меняет пространство возможных следующих шагов.

Семейство не сводится к одному оператору. Выражение $o_1 \cup o_2$ не является оператором: поскольку эффект o_2 зависит от того, применён ли уже o_1 , не существует

функции только от исходного состояния Σ , которая схватывала бы эту пару.

Внешние операторы не гарантируют результата. o_{abd} может предложить противоречивую гипотезу, o_{anal} — неподходящее отображение, o_{int} — ошибочный ответ. Это не дефект, а природа доксы: она предлагает, но не гарантирует. Внутренние операторы, напротив, являются детерминированными функциями F и своего входа; протокольные операторы гарантируют детерминизм и проверяемость, а не истину.

Отметим также, что композиция операторов определена не всегда: некоторые пары операторов несовместимы (например, два оператора, каждый из которых требует монопольного доступа к Π). Идемпотентность отсутствует: повторное применение o_{abd} может породить новую гипотезу. Обратимость также отсутствует: отмена гипотезы меняет теорию F , но запись о ней остаётся в журнале $\mathcal{J}$. Полное исследование алгебраических свойств семейства — тема отдельной работы; здесь мы фиксируем лишь те свойства, которые существенны для дальнейшего.

3.8. Процесс рассуждения

Рассуждение — это *последовательность* расширений:

$$\Sigma_0 \xrightarrow{o_1} \Sigma_1 \xrightarrow{o_2} \Sigma_2 \xrightarrow{o_3} \dots$$

где каждый шаг — *акт* обращения к доксе, помеченный оператором, который его опосредует; фиксацию, классификацию и применение каждого акта выполняет протокол. Не «вычисление», а *обращение* к источнику.

Каждый шаг *фиксируется* в журнале: идентификатор оператора, волна, обоснование, источник и результат (принято / отвергнуто / отложено). Журнал — это *протокол* взаимодействия доксы и логоса. Он *не является* частью ни доксы, ни логоса. Он — *интерфейс*, протокольный оператор, а не функция доксы.

3.9. Границы модели

Список операторов *открыт*: модель не утверждает, что выделены все. Могут существовать операторы, которых мы не видим, и аспекты доксы, которые через операторы не выражаются. Это не временное ограничение, а следствие предмета: докса не является теорией в L , и её взаимодействие с логосом описывается ровно теми функциями, которые удалось выделить.

3.10. Роль логоса

Логос не пассивен. Он классифицирует каждое предложение доксы:

- **derivable** — предложение уже выводится из F . Принимается как *нарратив*.
- **cited** — предложение заземлено *цитатой* из текста. Добавляется в аксиомы.
- **hypothesis** — предложение ново. Записывается в журнал с меткой H .
- **rejected** — предложение не проходит проверку. Отбрасывается с кодом причины.

Классификация — это протокольная функция; вердикт, который она выносит, принадлежит логосу. Классификация *детерминирована*: она зависит от *структуры F* и *содержания* предложения, а не от «настроения» логоса. Это *гарантия* того, что логос не «поддаётся» доксе. Он *принимает* только то, что *может* принять. И *честно* говорит, когда *не может*.

4. Реализация: Ankyra

В разделе 3 мы ввели семейство операторов взаимодействия доксы и логоса. Здесь мы покажем, что эта модель не является теоретической конструкцией: она реализована в работающей системе, гибридной нейро-символической системе рассуждений Ankyra [21]. Мы опишем, как реализована каждая из трёх ролей — внешние операторы, внутренние операторы и протокол, — и отметим, что полное описание архитектуры и её эмпирическая оценка — предмет отдельной работы [22].

4.1. Докса — это LLM

Что именно LLM приносит в систему в роли доксы. Она обучена на текстах, а текст — это не просто средство передачи информации, а картина мира: грамматика, лексика и метафоры задают сетку, через которую реконструируется реальность. LLM — гипертрофированная версия этой сетки. Она несёт в себе сжатый человеческий опыт: культурное поле, доступное через предсказание следующего токена; историчность, застывшую в вероятностях. Именно это делает её носителем доксы — не вычислимость и не архитектура, а то, что она есть *слепок* человеческого языка и опыта.

4.2. Принцип: LLM предлагает, движок решает

Ядро Ankyra — различение двух участников. LLM выполняет роль доксы: читает текст, разбирает его на структуру и в цикле рассуждений предлагает дополнительные шаги. Символьный движок выполняет роль логоса: превращает структуру в формальную теорию, достраивает её выводом, сопоставляет с вопросом и удостоверяет то, что действительно следует.

Граница абсолютна. LLM никогда не владеет истиной. В теорию не попадает ничего, что движок не сможет обосновать либо дословной цитатой из текста, либо явной пометкой гипотезы.

4.3. Контракт предложения

Каждое утверждение, которое LLM хочет добавить, классифицируется детерминированно в одну из четырёх категорий: *derivable* (уже следует из теории), *cited* (подкреплено дословной цитатой), *hypothesis* (знание, которого в тексте нет), *rejected* (не проходит проверку). Это прямая реализация оператора порождения гипотез (o_{abd}): LLM предлагает, движок классифицирует, ничто не принимается без основания. Различение между *cited* и *hypothesis* делает ответы честными: вывод на цитатах — *proven*, вывод на предположениях — *proven_under(H)* с перечислением этих предположений. Ответ никогда не бывает сильнее стоящих за ним оснований.

4.4. Цикл рассуждений

Задача проходит четыре фазы: *фаза 0* — декомпозиция (LLM разбирает текст на структуру), *фаза 1* — решение (движок достраивает теорию), *фаза 2* — цикл с подсказками (LLM предлагает по одному шагу за раз, если теория не решает вопрос), *фаза 3* — объяснение (цепочка собирается механически из истории доказательства).

Цикл с подсказками (фаза 2) организован в *волны*. Волна — это один полный акт обращения к доксе: движок проверяет текущее состояние теории и формулирует пробелы, LLM получает подсказку с этими пробелами и предлагает *ровно одно* типизированное предложение, движок классифицирует это предложение и применяет или отбрасывает его, после чего волна фиксируется в журнале и цикл начинается заново. Ограничение «одно предложение за волну» принципиально: оно делает каждое обращение к доксе проверяемым и оставляет след, по которому можно восстановить, на каком именно шаге и почему изменился ответ.

Цикл с подсказками реализует процесс обращения к доксе из раздела 3: каждая

волна — это один акт $F_i \xrightarrow{o} F_{i+1}$, опосредованный *внешним* оператором, тогда как фиксацию, классификацию и применение выполняет протокол.

4.5. Журнал

Каждый шаг фиксируется в журнале: идентификатор оператора, волна, обоснование, источник, результат. Журнал — это протокол взаимодействия доксы и логоса, не часть ни того, ни другого. Он реализует *протокольную* роль семейства (журнал и фиксация): фиксирует каждый акт обращения к доксе и делает процесс проверяемым и воспроизводимым. Это не функция доксы.

4.6. Монотонная база и движущийся ответ

База — аксиомы, правила, гипотезы — только растёт: посылки не удаляются, цель не ослабляется. Но ответ может меняться по мере поступления новой информации. Каждое изменение записывается как Revision с указанием волны и причастных предположений. Это *внутренний* оператор семейства (ревизия, o_{rev}): докса поставляет информацию, логос вычисляет, как она влияет на теорию, и решение фиксируется, а не скрывается.

4.7. Решатели и объявленный фрагмент

Ankura — не монолитный решатель, а оркестратор формализаций и решающих процедур. Разные виды задач требуют разных логик: хорновские клаузы (L0), стратифицированное отрицание (L1), дизъюнкция и кванторы (L2), конечнодоменные ограничения (L3), арифметика (L4), умолчания со специфичностью (D). Эти решатели — *внутренние* операторы семейства (логосная сторона): L1 и D реализуют соответственно отмену (o_{def}) и специфичность (o_{spec}), а L2 реализует разбор случаев внутри решающей процедуры. Ни один из них не является источником содержания.

Выбор процедуры осуществляется через контракт объявленного фрагмента: движок исследует построенные теорию и запрос и выводит набор требуемых признаков. Если фрагмент укладывается в доступные возможности — выбирается соответствующая процедура; если нет — результатом становится именованный пробел `out_of_fragment`, а не тихое понижение до более слабой логики. Объявление фрагмента — это *протокольная* функция; его применение внутренне, и система честно сообщает о несоответствии, а не угадывает.

4.8. Арбитр в реализации

В Ankura роль Арбитра разделена. Автоматический Арбитр — это *тестовый раннер*: он возвращает объективный вердикт (тест прошёл или нет) и не зависит от предложений доксы. Пользовательский Арбитр — это *человек*, который принимает задачи, задаёт цели и решает, когда процесс завершён. Автоматический Арбитр подключается в фазе 1 (когда проверяется, следует ли цель) и в фазе 2 (когда предложение доксы изменяет теорию); пользовательский Арбитр — во внешнем интерфейсе системы. Полное описание роли Арбитра в архитектуре — предмет отдельной работы [22].

4.9. Честность по построению

Ценность системы держится на гарантиях, обеспечиваемых механически: в теории нет фактов, принадлежащих модели; вопрос никогда не является источником; в детерминированном коде нет разбора естественного языка; предположив P , нельзя опровергнуть $\neg P$; ответ, использовавший предположения, всегда помечен `proven_under (H)`. Это реализация свойства прозрачности интерфейса: логос принимает только то, что может принять, и честно говорит, когда не может.

4.10. Связь с концептуальной моделью

Ankura реализует три роли семейства из раздела 3. *Внешние*: контракт предложения (оператор порождения, o_{abd}). *Внутренние*: ревизия (учёт `Revision`), отмена (L1), специфичность (D). *Протокольные*: журнал, детерминированная классификация, проверки заземления цитат, волновой протокол и объявление фрагмента. Роль *Арбитра* реализована как композиция автоматического (тестовый раннер) и пользовательского (человек) компонентов. Два внешних оператора пока не реализованы: аналогия (o_{anal}) и интуиция (o_{int}). Система не претендует на полноту: она реализует то, что может реализовать честно, и честно говорит о том, чего не может.

5. Обсуждение

Мы ввели различие доксы и логоса, показали его формальные аналоги и предложили семейство операторов взаимодействия доксы и логоса, реализованное в Ankura. Теперь необходимо обсудить границы этого подхода: что он объясняет, чего не объясняет, и какие следствия из него вытекают.

5.1. Что модель объясняет

Модель показывает, каким образом формальная система может расширяться. Теоремы Гёделя и Тьюринга описывают пределы замкнутых систем. Наша модель показывает, что открытая система — та, которая взаимодействует с доксой через явный интерфейс, — может расширяться без того, чтобы это расширение было выводом внутри неё самой. Расширение приходит извне. Логос его не производит. Он его принимает, отвергает или откладывает.

Неполнота не устраняется, а *переносится*: полученная теория F' снова неполна, если она непротиворечива и достаточно богата, но точка, в которой система останавливается, сместилась с замыкания на интерфейс. Система продолжает работать там, где замкнутая система остановилась бы.

5.2. Границы подхода

Модель работает с функциями взаимодействия, а не с содержанием доксы. Это граница предмета, а не временное ограничение: докса не является теорией в языке L , и описывать в ней нечего — кроме тех операций, через которые она расширяет формальную систему. Именно эти операции и составляют предмет модели.

Модель также не гарантирует, что набор операторов, который мы выделили, полон. Мы рассматриваем его как открытый список. Операторы аналогии и интуиции, не реализованные в текущей версии, показывают, что пространство операторов шире того, что мы описали. Могут существовать операторы, которых мы не видим. Это не недостаток модели, а следствие её предмета.

Наконец, модель не даёт критерия «правильного» выбора между конкурирующими предложениями доксы. Если два оператора порождают несовместимые гипотезы, система фиксирует конфликт, но не решает его автоматически. Выбор остаётся за *Арбитром* (раздел 3) — человеком или внешней программой, действующей по объявленному критерию. Модель намеренно не автоматизирует этот шаг: никакой формальный критерий не решает спор между несовместимыми предложениями доксы, если оба совместимы с теорией.

5.3. Связь с оракулом

В теории вычислений понятие оракула было введено Тьюрингом для описания машины, которая может получать ответы на вопросы, не решаемые её собственной процедурой [23]. Оракул не является частью машины, но машина может к нему обращаться. Докса в нашей модели играет структурно близкую роль.

Однако есть важное различие. Оракул даёт *истинные* ответы на вопросы определённого класса. Докса не даёт истинных ответов: она даёт *предложения*, которые могут быть истинными, ложными или неопределёнными. Именно поэтому интерфейс к доксе включает классификацию — предложение входит в систему не как истина, а как гипотеза, требующая проверки. Логос не доверяет доксе. Он её использует.

5.4. Практические следствия

Для объяснимого искусственного интеллекта модель даёт один ясный принцип: *ответ не должен быть сильнее своих оснований*. Если система вывела заключение из цитат — это одно. Если из предположений — другое. Если из смеси — третье, и это третье должно быть явно помечено. Пользователь имеет право знать, на чём стоит ответ.

Для верификации систем это означает, что проверяемость не требует полной формализации. Достаточно, чтобы каждый шаг был прослеживаем до источника, а каждое расширение фиксировалось в журнале. Докса остаётся неформализуемой, но её *след* в системе — формален.

Для архитектуры гибридных систем это означает, что разделение между нейросетевой и символьной частью не является инженерным компромиссом. Это структурное разделение, отражающее различие доксы и логоса. Нейросетевая часть не «слабее» символьной. Она делает то, что символьная делать не может. Символьная часть не «жестче» нейросетевой. Она делает то, что нейросетевая не должна. Каждая сторона сильна в своём.

5.5. Что остаётся открытым

Модель оставляет открытыми несколько вопросов. Как соотносятся операторы разных ролей — существуют ли законы, связывающие внешние, внутренние и протокольные шаги, тождества, приоритеты? Как измерить эффективность оператора — по глубине изменения теории, по числу принятых предложений, по вкладу в итоговый ответ? Как отличить оператор, который действительно расширяет систему, от оператора, который лишь переформулирует уже известное? Эти вопросы не имеют ответа в рамках текущей работы. Они указывают направление дальнейшего исследования.

6. Связанные работы

6.1. Карта области

Систематический обзор 2026 года [24] (178 работ, 2020–2025) выделяет четыре устойчивых паттерна нейросимволической интеграции: последовательный каскад, символический валидатор, символический планировщик с нейронным исполнителем и дифференцируемое объединение. Ankyra ближе всего к последовательному каскаду с ограниченным циклом, но отличается от него двумя вещами: протоколом предложений, при котором каждое утверждение входит в теорию только с цитатой или пометкой допущения, и объявленным фрагментом, при котором несовпадение требуемой и доступной логики даёт честный `out_of_fragment`, а не скрытый переход к более слабой процедуре. Тот же обзор отмечает, что мета-когнитивные способности — мониторинг собственных выводов, распознавание нехватки знаний, переключение режимов — встречаются лишь в 5% работ; `out_of_fragment` и `proven_under(H)` можно рассматривать как вклад именно в эту нишу.

Ближайшая по духу система — *Autonomous Trustworthy Agents* (ATA) [25]: она также добивается детерминизма и проверяемости, но достигает этого *полным выносом* LLM за пределы runtime (LLM строит базу знаний офлайн, онлайн работает только символический решатель). Ankyra сохраняет LLM в цикле, ограничивая его протоколом; это принципиально разные точки на одном спектре. Тот же подход к верификации регуляторных документов через SMT-солвер описан в [26].

6.2. Языковая модель и символический решатель

Конвейеры вида «переводчик + решатель» (Logic-LM [27], LINC [28]) также передают рассуждение символическому решателю, но доверяют переводу целиком: если формализация неверна, неверно и заключение. В нашей модели граница строже: каждое утверждение входит в теорию только с дословной цитатой или явной пометкой допущения, обоснованность ответа выводится из его посылок (`proven` против `proven_under`), а несовпадение требуемой и доступной логики даёт честный `out_of_fragment`, а не скрытый переход к более слабой процедуре. Эмпирическое сравнение Ankyra с Logic-LM и LINC на общем входе приведено в сопутствующей работе [22].

6.3. Немонотонные рассуждения и умолчания

Логика умолчаний [9] и циркумскрипция [10] формализуют отменяемость выводов; Ankyra реализует слой умолчаний со специфичностью через `is_a` и честно сообщает о неразрешённом конфликте, не угадывая победителя. Философская рамка (докса/логос), введённая в этой работе, отличается от чисто инженерных подходов к разделению LLM и решателя: она выводит архитектурный принцип из эпистемологического различия, восходящего к античной традиции.

6.4. История вывода и проверяемость

Опора на автоматически собранную историю вывода сближает Ankyra с работами о проверяемом выводе. Объяснение здесь — изложение доказательства, а не второе мнение о нём: модель не вправе добавлять в него факты, а каждый шаг называет сработавшее правило, использованные связывания и обосновывающую цитату или гипотезу.

7. Заключение

Мы предложили интерпретацию теорем о неполноте не как абсолютных границ познания, а как границ статических систем. Человеческое мышление не является статической системой: оно развивается, опираясь на доксу — неформализуемый источник жизненного опыта, культурного поля и телесного знания. Различение доксы и логоса имеет формальные аналоги в современной логике, но ни один из них не формализует доксу как таковую.

Наш вклад состоит в семействе операторов взаимодействия доксы и логоса — формальном интерфейсе между источником предложений и формальной системой, — том, что этот интерфейс *асимметричен*: внешние операторы поставляют содержание, внутренние решают, а протокольные делают обмен проверяемым. Мы реализовали эту модель в системе Ankyra, где большая языковая модель выполняет роль доксы, а символьный решатель — роль логоса [21]. Принцип «LLM предлагает, движок решает» обеспечивает прозрачность: ничто не попадает в теорию без обоснования, и ответ никогда не бывает сильнее стоящих за ним оснований.

Мы не утверждаем, что предложили решение проблемы неполноты. Мы утверждаем, что показали, как система может работать с этой проблемой, не пытаясь её преодолеть. Докса не отменяет логос — она его питает. И в этом взаимодействии не поражение формализации, а её расширение.

Список литературы

- [1] Gödel K. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I // Monatshefte für Mathematik und Physik. 1931. Bd. 38. S. 173–198.
- [2] Turing A. M. On Computable Numbers, with an Application to the Entscheidungsproblem // Proceedings of the London Mathematical Society. 1936. Ser. 2, Vol. 42. P. 230–265.
- [3] Diels H., Kranz W. Die Fragmente der Vorsokratiker. Berlin: Weidmann, 1903–1910.
- [4] Платон. Тезтет // Платон. Собрание сочинений: в 4 т. М.: Мысль, 1990–1994. Т. 2.
- [5] Аристотель. Никомахова этика // Аристотель. Собрание сочинений: в 4 т. М.: Мысль, 1975–1983. Т. 4.
- [6] Hintikka J. Knowledge and Belief: An Introduction to the Logic of the Two Notions. Ithaca: Cornell University Press, 1962.
- [7] Segerberg K. The Basic Dynamic Doxastic Logic of AGM // Frontiers in Belief Revision / Ed. by M.-A. Williams, H. Rott. Dordrecht: Kluwer, 2001. P. 57–84.
- [8] Segerberg K. Some Completeness Theorems in the Dynamic Doxastic Logic of Iterated Belief Revision // The Review of Symbolic Logic. 2010. Vol. 3, No. 1. P. 1–24.
- [9] Reiter R. A Logic for Default Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 81–132.
- [10] McCarthy J. Circumscription—A Form of Non-Monotonic Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 27–39.
- [11] Moore R. C. Semantical Considerations on Nonmonotonic Logic // Artificial Intelligence. 1985. Vol. 25, No. 1. P. 75–94.
- [12] Пирс Ч. С. Начала прагматизма. СПб.: Алетейя, 2000.
- [13] Peirce C. S. Lectures on Pragmatism (1903) // The Collected Papers of Charles Sanders Peirce. Cambridge: Harvard University Press, 1931–1958. Vol. 5.
- [14] Niiniluoto I. Defending Abduction // Philosophy of Science. 1999. Vol. 66, No. 3. P. S436–S451.

- [15] Kakas A. C., Kowalski R. A., Toni F. Abductive Logic Programming // Journal of Logic and Computation. 1992. Vol. 2, No. 6. P. 719–770.
- [16] Виньков М. М., Фоминых И. Б. Темпоральные немонотонные логические системы: взаимосвязи и вычислительная сложность // Искусственный интеллект и принятие решений. 2008. № 4. С. 19–25.
- [17] Cadoli M., Schaerf M. A Survey of Complexity Results for Non-Monotonic Logics // Journal of Logic Programming. 1993. Vol. 17, No. 2–4. P. 127–160.
- [18] Eiter T., Gottlob G. Propositional Abduction is Σ_2^P -Complete // Information Processing Letters. 1995. Vol. 55, No. 4. P. 213–218.
- [19] Eiter T., Gottlob G. The Complexity of Logic-Based Abduction // Journal of the ACM. 1995. Vol. 42, No. 1. P. 3–42.
- [20] Alchourrón C. E., Gärdenfors P., Makinson D. On the Logic of Theory Change: Partial Meet Contraction and Revision Functions // The Journal of Symbolic Logic. 1985. Vol. 50, No. 2. P. 510–530.
- [21] Ankyra: A domain-agnostic neuro-symbolic reasoning engine. GitHub repository. URL: <https://github.com/merfill/ankyra>
- [22] Лапшин В. Ankyra: корректное нейросимволическое рассуждение с проверяемым выводом. (сопутствующая работа, в подготовке).
- [23] Turing A. M. Systems of Logic Based on Ordinals // Proceedings of the London Mathematical Society. 1939. Ser. 2, Vol. 45. P. 161–228.
- [24] Bin Hakim S., Adil M., Velasquez A., Song H. H. Neuro-symbolic agentic AI: Architectures, integration patterns, applications, open challenges and future research directions // Computer Science Review. 2026. Vol. 60. Art. 100902. DOI: 10.1016/j.cosrev.2026.100902.
- [25] Peer D., Stabinger S. ATA: Autonomous Trustworthy Agents. arXiv:2510.16381.
- [26] Hsia Y.-S., Yu F., Jiang J.-H. R. Neuro-Symbolic Compliance: LLM-driven SMT Verification of Regulatory Documents. arXiv:2601.06181.
- [27] Pan L. et al. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. Findings of EMNLP. 2023.
- [28] Olausson T. X. et al. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. EMNLP. 2023.

- [29] ДСМ-метод автоматического порождения гипотез: Логические и эпистемологические основания / сост. О. М. Аншаков, Е. Ф. Фабрикантова; под общ. ред. О. М. Аншакова. М.: URSS, 2009. 430 с.
- [30] Финн В. К. Эпистемологические основания ДСМ-метода автоматического порождения гипотез // Вопросы философии. 2014. № 2. С. 83–96.
- [31] Finn V. K. Epistemological foundations of the JSM method for automatic hypothesis generation // Automatic Documentation and Mathematical Linguistics. 2014. Vol. 48, No. 2. P. 96–148.